

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-a59a249ffad0190f4.jsonl`  
- SHA-256: `eb705a300098a526154d6a49880f78efec03c51feb54dd0d48113a4e3d56b25a`  
- Source modified: `2026-07-06T19:19:48+00:00`  
- Imported at: `2026-07-08T16:00:11+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:18:19.571Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
 - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (design): Tier 2/3 caption/keyword matching depends on ContentItem.text, which is NULL in the live path ? an unstated hard dependency
severity high | section Decision ? Topic preferences (Tier 2); Security stages (Tier 3, caption text scanning)
summary: Tier 2 'message caption keyword match' and Tier 3 'caption text scanning' / 'URL/domain reputation (check links in captions)' all read caption text, but ContentItem.text is documented NULL until a later track populates it, and nothing on main writes it ? including the live handler's own insert path.
why_it_matters: The live handler (telethon_client.py) ingests via the VideoRecord/insert_video shim, which constructs ContentItem WITHOUT text= (db.py ~1620), so text is NULL for every live item. extract_video_meta never reads message.message (the caption) at all. So F2 keyword-on-caption and F4 link-extraction-from-caption have no data to match against. This is a cross-track dependency the ADR doesn't call out: Track F needs a prior/companion change to populate ContentItem.text from Message.message in the ingestion path. Without it F2/F4 are no-ops.
evidence: models.py ContentItem.text docstring: 'nullable in Phase 1', 'stay NULL until later tracks populate them'. `git grep text=` finds no caller passing caption text to ContentItem; the insert_video shim omits text. telethon_client.py extract_video_meta reads only Document attributes, never message.message.
rec: Add an explicit dependency/pre-req to the phasing table: 'F2/F4 require ContentItem.text populated from the message caption in the ingestion hot path (new work, not delivered by Track B/C).' Name which track/PR lands caption population, and make F2/F4 depend on it ? not just on F1.

2. assistant (2026-07-06T19:18:22.097Z)

I'll verify this finding by examining the ADR and the referenced code.

3. user (2026-07-06T19:18:23.697Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

Concept	Current state
FilterConfig / Policy	One JSON blob per account. Flat duration/resolution/size/exclude rules. AND semantics.

```
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig` blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffmpeg were deferred to v2 in the original plan. |
```

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?                               Policy Pipeline                       ?
?                               ?                                     ?
? Tier 1 ? Metadata gates (always runs, zero I/O)                   ?
?   duration, resolution, size, codec, mime, GIF/round              ?
?   ? extends today's FilterEngine                                  ?
?                               ?                                     ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)                ?
?   source allowlist, keyword match, channel topic tags            ?
?   ? user-expressed interests filter the stream                   ?
?                               ?                                     ?
? Tier 3 ? Content safety (opt-in, metadata + light I/O)            ?
?   URL/domain reputation, file metadata sanity,                   ?
?   caption text scanning                                           ?
?   ? cheap checks; no download                                     ?
?                               ?                                     ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)                      ?
?   ffmpeg, perceptual hash, downloaded content scan              ?
?   ? gated behind explicit user opt-in + budget                   ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"
 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI
```

```
class PolicyStage(Protocol):
 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
 async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
 ...
```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult`` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json`` under a `topics`` key:

```
```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```
```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist`` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint`` across accounts (already handled by `content_items.dedup_key``; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata`` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}``).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated** behind explicit opt-in because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true`` in `config_json``) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
```

```

    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
...

```

The caller (telethon_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

Schema ? minimal changes

****No schema migration**** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

Implementation phasing (future issues)

Phase	Scope	Dependencies
---	---	---
F1	Policy engine prototype	`PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler None
F2	Topic preferences	`topics` config parsing, source title/caption keyword matching F1
F3	Source allowlist	`allowlist` config, skip non-allowlisted sources F1
F4	URL/domain reputation	Domain blocklist, caption link extraction F1
F5	Deep inspection	ffprobe/perceptual hash, download budget enforcement F1, ADR 0003 waiver
F6	Policy decisions table	Audit trail storage, explainability UI F1
F7	Explainability endpoint	`/why <content_item_id>` command showing the explanation trace F6

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the ****design contract**** the phases code against.

Consequences

****Positive****

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

****Negative****

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same

- interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

Alternatives considered

- ****Do nothing; keep flat FilterConfig.**** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- ****Inline each stage in the handler**** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- ****Use a rules engine / DSL.**** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- ****Bundle all phases into a single PR.**** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

Open questions for the decision owner

1. ****`StageResult.metadata` shape**** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. ****Policy decisions table**** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. ****Topic matching**** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. ****Deep inspection budget**** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

References

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

4. assistant (2026-07-06T19:18:27.231Z)

Let me verify the key claims by examining the code.

5. user (2026-07-06T19:18:29.273Z)

```
c4c4d18 fix(scanning): don't clobber user-controlled watched on source upsert (#41)
3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR
0009, Track E) (#39)
35f4f70 feat(commands): extract CommandService ? provider-independent command execution (ADR
0011, Track D) (#38)
---BRANCH---
```

main

6. user (2026-07-06T19:18:30.280Z)

```
26: - ``LINK``: send a text message with a t.me link + metadata summary.
153: (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
169: text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
```

7. user (2026-07-06T19:18:34.470Z)

```
140 kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141 watched: bool = Field(
142     default=False,
143     description="ADR 0009 selective scanning: ingest from this source iff True",
144 )
145 seen_first_at: datetime = Field(default_factory=_utcnow)
146 last_seen_at: datetime | None = None
147
148
149 class ContentItem(BaseModel):
150     """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152     Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153     (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154     NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155     ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156     wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157     """
158
159     model_config = ConfigDict(frozen=True)
160
161     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162     account_id: int = Field(description="FK ? accounts.id")
163     source_id: int = Field(description="FK ? sources.id")
164     provider: str = Field(default="telegram", description="Provider namespace")
165     provider_message_id: str = Field(
166         description="Provider-native message id (Telegram message_id, ?)"
167     )
168     content_type: str = Field(default="video", description="'video' | 'link' |
'article'")
169     text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170     link_url: str | None = Field(default=None, description="For link/article posts
(Phase 1: NULL)")
171     media_ref: str | None = Field(default=None, description="Provider media reference")
172     document_id: str | None = Field(
173         default=None,
174         description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175     )
176     duration_s: int | None = None
177     width: int | None = None
178     height: int | None = None
179     size_bytes: int | None = None
180     mime_type: str | None = None
181     dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
per account")
182     matched: bool = False
183     seen_at: datetime = Field(default_factory=_utcnow)
184
185
186 class Policy(BaseModel):
187     """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189     Generalizes the legacy per-account ``filter_config`` row so one account can
```

8. user (2026-07-06T19:18:36.002Z)

```
src/tg_video_filter/backfill.py:250: link_text=link_text,
src/tg_video_filter/db.py:865: text=row["text"],
src/tg_video_filter/telethon_client.py:313: link_text=link_text,
```

9. assistant (2026-07-06T19:18:40.107Z)

The only `text=` for ContentItem is at db.py:865 (reading a row back). Let me verify the insert path and whether extract_video_meta reads the caption.

10. user (2026-07-06T19:18:41.626Z)

```
840         )
841
842
843     def _row_to_source(row: aiosqlite.Row) -> Source:
844         return Source(
845             id=row["id"],
846             account_id=row["account_id"],
847             provider=row["provider"],
848             provider_source_id=row["provider_source_id"],
849             title=row["title"],
850             kind=row["kind"],
851             watched=bool(row["watched"]),
852             seen_first_at=row["seen_first_at"],
853             last_seen_at=row["last_seen_at"],
854         )
855
856
857     def _row_to_content_item(row: aiosqlite.Row) -> ContentItem:
858         return ContentItem(
859             id=row["id"],
860             account_id=row["account_id"],
861             source_id=row["source_id"],
862             provider=row["provider"],
863             provider_message_id=row["provider_message_id"],
864             content_type=row["content_type"],
865             text=row["text"],
866             link_url=row["link_url"],
867             media_ref=row["media_ref"],
868             document_id=row["document_id"],
869             duration_s=row["duration_s"],
870             width=row["width"],
871             height=row["height"],
872             size_bytes=row["size_bytes"],
873             mime_type=row["mime_type"],
874             dedup_key=row["dedup_key"],
875             matched=bool(row["matched"]),
876             seen_at=row["seen_at"],
877         )
878
879
```

11. user (2026-07-06T19:18:43.166Z)

```
858:     return ContentItem(
1620:     item = ContentItem(
```

12. user (2026-07-06T19:18:46.669Z)

```
1600
1601     async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
1602         """Deprecated shim: insert into ``content_items`` (ADR 0011).
1603
1604         Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
```



```

1605         if absent), computes the provider-neutral ``dedup_key`` via
1606         :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1607         ``VideoRecord`` shape is reconstructed from the inserted row.
1608         """
1609         src = await upsert_source(
1610             conn,
1611             Source(
1612                 account_id=video.user_id,
1613                 provider="telegram",
1614                 provider_source_id=str(video.channel_id),
1615                 kind="channel",
1616                 watched=False,
1617             ),
1618         )
1619         assert src.id is not None # upsert_source populates the surrogate PK
1620         item = ContentItem(
1621             account_id=video.user_id,
1622             source_id=src.id,
1623             provider="telegram",
1624             provider_message_id=str(video.message_id),
1625             content_type="video",
1626             document_id=str(video.document_id) if video.document_id is not None else None,
1627             duration_s=video.duration_s,
1628             width=video.width,
1629             height=video.height,
1630             size_bytes=video.size_bytes,
1631             mime_type=video.mime_type,
1632             dedup_key=compute_dedup_key(
1633                 provider="telegram",
1634                 document_id=(str(video.document_id) if video.document_id is not None else
None),
1635                 source_id=src.id,
1636                 provider_message_id=str(video.message_id),
1637                 size_bytes=video.size_bytes,
1638                 duration_s=video.duration_s,
1639                 height=video.height,
1640                 width=video.width,
1641             ),
1642             matched=video.matched,
1643             seen_at=video.seen_at,
1644         )
1645         item = await insert_content_item(conn, item)
1646         return _content_item_to_video(item, src, video.user_id, video.channel_id)
1647
1648
1649     async def mark_forwarded(conn: aiomysql.Connection, video_id: int) -> None:
1650         """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).
1651
1652         Per-destination delivery state moves to ``delivery_attempts`` (Track C,
1653         #28). For Phase 1 (before Track C lands) there is no equivalent column on
1654         ``content_items`` ? the ``matched`` flag alone is sufficient because v1

```

13. assistant (2026-07-06T19:18:49.557Z)

Confirmed: the live insert path (`insert\_video` shim, db.py:1620) constructs `ContentItem` WITHOUT `text=`, so it defaults to None. Now let me verify the live handler flow and that `extract\_video\_meta` never reads the caption (`message.message`).

14. user (2026-07-06T19:18:51.091Z)

```

280         user.id,
281         channel_id,
282         message_id,
283         document_id,
284     )

```

```

285         return
286     log.info(
287         "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
288         user.id,
289         channel_id,
290         message_id,
291         result.matched,
292         result.reasons,
293     )
294
295     if result.matched:
296         # Build link text for LINK/BOTH modes (None for FORWARD).
297         link_text = None
298         if delivery_mode != DeliveryMode.FORWARD:
299             channel_title = _channel_title(event)
300             link_text = build_link_text(
301                 message_link=getattr(message, "link", None),
302                 duration_s=meta.duration_s,
303                 height=meta.height,
304                 size_bytes=meta.size_bytes,
305                 channel_title=channel_title,
306             )
307         await deliver(
308             client,
309             message,
310             conn,
311             inserted.id,
312             delivery_mode,
313             link_text=link_text,
314             target=delivery_target,
315         )
316
317     return on_new_message
318
319
320 # --- helpers -----
321
322
323 def _extract_document_id(message: Message) -> int | None:
324     """Extract Telegram's ``document.id`` from a video message.
325
326     Returns None if the message has no document media or the id is absent.
327     This is the most reliable content-based dedup key: Telegram reuses the
328     same ``document.id`` when a file is forwarded across channels.
329     """
330     media = getattr(message, "media", None)
331     if isinstance(media, MessageMediaDocument):
332         media = media.document
333     if isinstance(media, Document):
334         return getattr(media, "id", None)
335     return None
336
337
338 def _peer_id(peer: object) -> int:
339     """Best-effort numeric id for the peer of a message.

```

15. user (2026-07-06T19:18:56.124Z)

```

6:         -> extract_video_meta(message) # pure, unit-tested
46: def extract_video_meta(message: Message) -> VideoMeta | None:
168:     message: Message = event.message # type: ignore[assignment]
176:     meta = extract_video_meta(message)
368:     "extract_video_meta",

```

16. user (2026-07-06T19:18:58.529Z)

```

46 def extract_video_meta(message: Message) -> VideoMeta | None:
47     """Extract ``VideoMeta`` from a Telethon ``Message``, or ``None`` if not a video.
48
49     Returns ``None`` when the message has no video/document media. Returns a
50     ``VideoMeta`` (with fields possibly ``None``) for a video whose individual
51     attributes are missing ? the filter engine decides how to treat gaps.
52
53     Telegram stores video metadata in two places:
54     - ``Document``: ``size`` (bytes), ``mime_type`` (str), ``attributes`` (list)
55     - ``DocumentAttributeVideo`` (inside ``attributes``): ``duration`` (float s),
56       ``w`` (width), ``h`` (height)
57
58     Note: we inspect ``message.media`` directly rather than the ``.video``
59     convenience property, because the latter only exists on Telethon's *custom*
60     Message wrapper (applied at runtime) and is ``None`` on the raw TL Message
61     used in tests. A Document is considered a video iff it contains a
62     ``DocumentAttributeVideo``.
63     """
64     media = getattr(message, "media", None)
65
66     # Real Telegram messages wrap documents in MessageMediaDocument.
67     # Unwrap it to get the actual Document.
68     if isinstance(media, MessageMediaDocument):
69         media = media.document
70
71     if not isinstance(media, Document):
72         return None
73
74     # Only treat this as a video if the document carries a video attribute.
75     has_video_attr = any(isinstance(a, DocumentAttributeVideo) for a in
media.attributes)
76     if not has_video_attr:
77         return None
78
79     meta = VideoMeta(
80         size_bytes=media.size,
81         mime_type=media.mime_type or None,
82     )
83
84     # Find the video attribute for duration/dimensions and round/nosound flags.
85     video_update: dict = {}
86     for attr in media.attributes:
87         if isinstance(attr, DocumentAttributeVideo):
88             if attr.duration:
89                 video_update["duration_s"] = int(round(attr.duration))
90             if attr.w:
91                 video_update["width"] = attr.w
92             if attr.h:
93                 video_update["height"] = attr.h
94             video_update["is_round"] = attr.round_message # may be None
95             video_update["nosound"] = attr.nosound # may be None
96             break
97
98     # Check for GIF (DocumentAttributeAnimated present).
99     is_gif = any(isinstance(a, DocumentAttributeAnimated) for a in media.attributes)
100
101     meta = meta.model_copy(update={**video_update, "is_gif": is_gif})
102     return meta
103
104
105 # --- client factory -----
106
107
108 def build_client(
109     session: TelegramSession, settings: SettingsLike

```

```

110 ) -> TelegramClient: # pragma: no cover
111     """Build a Telethon client with a file-backed SQLiteSession (ADR 0010).
112
113     The session path is read from the ``telegram_sessions.session_path`` column
114     (data-driven, no longer derived from the PK at runtime) and resolved
115     against ``settings.sessions_dir``. The directory is created if missing.
116     """
117     sessions_dir = Path(settings.sessions_dir)
118     sessions_dir.mkdir(parents=True, exist_ok=True)
119     # session_path is stored as a basename (e.g. "user_111.session"); join with
120     # the runtime sessions_dir so the migration did not need to know the path.
121     session_path = sessions_dir / session.session_path
122
123     client = TelegramClient(
124         str(session_path),
125         api_id=settings.tg_api_id,
126         api_hash=settings.tg_api_hash,
127     )
128     return client
129
130
131 class SettingsLike(Protocol):
132     """Structural type for the subset of Settings the client factory needs.
133
134     Using a Protocol keeps the module importable in tests without dragging in
135     the real Settings class. At runtime ``config.Settings`` satisfies this shape.
136     """
137
138     tg_api_id: int
139     tg_api_hash: str
140     sessions_dir: str
141
142
143 # --- event handler wiring -----
144
145
146 def make_new_message_handler(
147     *,
148     user: User,
149     conn: aiosqlite.Connection,
150     engine_holder: dict[str, FilterEngine],
151     client: TelegramClient,
152     delivery_mode: DeliveryMode = DeliveryMode.FORWARD,
153     delivery_target: str = "me",
154 ) -> _NewMessageCallback:
155     """Return an async callback for ``events.NewMessage``.

```

17. assistant (2026-07-06T19:19:03.125Z)

Confirmed. `extract\_video\_meta` reads only `message.media` (Document attributes) ? never `message.message` (the caption). The handler builds a `VideoRecord` from `meta` (no caption), then `insert\_video` builds `ContentItem` without `text`. Let me confirm the FilterConfig extra-claims claim and that the ingest path doesn't populate text via VideoRecord.

18. user (2026-07-06T19:19:04.493Z)

```

176     meta = extract_video_meta(message)
177     if meta is None:
178         return # not a video
179
180     channel_id = _peer_id(message.peer_id)
181     message_id = message.id
182     document_id = _extract_document_id(message)
183
184     # --- watched gate (ADR 0009, Track E) -----

```

```

185     # Opt-in: a source is ingested only when watched=True. An absent
186     # sources row means the channel has never been seen ? treat it as
187     # unwatched and lazily create a watched=0 row for /channels
188     # discoverability (ADR 0009 lines 37-44, 130-131).
189     if not await db.should_ingest_source(conn, user.id, channel_id):
190         return # skip unwatched / unknown source
191
192     # --- three-tier dedup (cheapest first) ---
193
194     # Tier 0: same (user, channel, message) instance ? already processed.
195     existing = await db.get_video(conn, user.id, channel_id, message_id)
196     if existing is not None:
197         return
198
199     # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
200     if document_id is not None:
201         existing = await db.get_video_by_document_id(conn, user.id, document_id)
202         if existing is not None:
203             log.info(
204                 "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
205                 "original_channel=%s original_msg=%s",
206                 user.id,
207                 document_id,
208                 channel_id,
209                 message_id,
210                 existing.channel_id,
211                 existing.message_id,
212             )
213             return
214
215     # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
216     if (
217         meta.size_bytes is not None
218         and meta.duration_s is not None
219         and meta.height is not None
220         and meta.width is not None
221     ):
222         existing = await db.get_video_by_content_fingerprint(
223             conn,
224             user.id,
225             meta.size_bytes,
226             meta.duration_s,
227             meta.height,
228             meta.width,
229         )
230         if existing is not None:
231             log.info(
232                 "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
233                 "channel=%s msg=%s original_channel=%s original_msg=%s",
234                 user.id,
235                 meta.size_bytes,
236                 meta.duration_s,
237                 meta.height,
238                 meta.width,
239                 channel_id,
240                 message_id,
241                 existing.channel_id,
242                 existing.message_id,
243             )
244             return
245

```

19. user (2026-07-06T19:19:14.893Z)

```

246         # Lazily record the channel (best-effort; failures are non-fatal).
247         try:
248             await db.upsert_channel(
249                 conn,
250                 Channel(user_id=user.id, channel_id=channel_id,
251                     title=_channel_title(event)),
252             )
253         except Exception:
254             log.warning(
255                 "failed to upsert channel %s for user %s", channel_id, user.id,
256             )
257         exc_info=True
258         result = engine_holder[ENGINE_KEY].evaluate(meta)
259         record = VideoRecord(
260             user_id=user.id,
261             channel_id=channel_id,
262             message_id=message_id,
263             document_id=document_id,
264             duration_s=meta.duration_s,
265             width=meta.width,
266             height=meta.height,
267             size_bytes=meta.size_bytes,
268             mime_type=meta.mime_type,
269             matched=result.matched,
270         )
271         try:
272             inserted = await db.insert_video(conn, record)
273         except sqlite3.IntegrityError:
274             # Lost a dedup race: a concurrent delivery of the same file
275             # (same document_id or same message instance) won the insert.
276             # The three-tier check above already classified this as new, so
277             # the collision means another task is processing (or has
278             # processed) it ? log and skip.
279             log.info(
280                 "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
281                 user.id,
282                 channel_id,
283                 message_id,
284                 document_id,
285             )

```

20. assistant (2026-07-06T19:19:19.283Z)

Fully confirmed. `VideoRecord` at line 258 carries no caption; `insert\_video` builds `ContentItem` with no `text`. The caption (`message.message`) is never read anywhere in the ingest path. Let me quickly confirm the FilterConfig extra-keys behavior and check VideoRecord has no text field.

21. user (2026-07-06T19:19:23.570Z)

```

FilterConfig extra keys OK -> dict_keys(['min_duration_s', 'max_duration_s', 'target_height',
'max_size_mb', 'exclude_gif', 'exclude_round', 'exclude_nosound', 'delivery_mode',
'delivery_target'])
extra config: None
VideoRecord fields: ['id', 'user_id', 'channel_id', 'message_id', 'document_id', 'duration_s',
'width', 'height', 'size_bytes', 'mime_type', 'matched', 'forwarded', 'seen_at']

```

22. assistant (2026-07-06T19:19:34.239Z)

All claims verified. `VideoRecord` has no `text`/caption field, so nothing in the live ingest path can carry a caption. `extract\_video\_meta` never touches `message.message`. `insert\_video` builds `ContentItem` without `text`. Thus `content\_items.text` is NULL for every live item.

Now the adversarial angle ? can I refute it as out-of-scope for a design doc? The ADR explicitly makes claims that its Tier 2/3 stages match against caption text: "message caption keyword match" (Tier 2), "caption text scanning" (Tier 3), "check links in captions" (Tier 3 URL/domain), and F4 "caption link extraction." The phasing table lists F2 deps=F1, F4 deps=F1 ? with no dependency on any caption-population work. The ADR's own References note ContentItem is the PolicyContext input, and models.py documents `text` stays NULL "until later tracks populate them." Nothing on main populates it, and no F-phase or named track owns that work. So F2/F4 as specified would match against NULL and be no-ops. This is a genuine unstated cross-track dependency.

The one calibration nuance: severity. This is a draft design doc, and the gap doesn't cause data loss ? it's a missing prerequisite that would make two future phases (F2, F4) no-ops if implemented against the design as written. Per the task's calibration guidance ("false claim/data-loss = high, gap = medium"), this is a design gap / missing dependency, not a false factual claim (the ADR never asserts text is populated; it just silently relies on it). That lands at medium.

23. user (2026-07-06T19:19:48.235Z)

Structured output provided successfully